

Can a Transformer Learn a Teacher’s Board and Voice Together?

Building a stroke+speech dataset from 1,191 lecture videos, and measuring whether the two modalities help each other

Taehyeon Kim

September 2026

Abstract

A math teacher on YouTube writes on the board and talks at the same time. I wanted to know whether a model can learn both from video — and, more specifically, whether the writing helps predict the speech and the speech helps predict the writing. To find out, I built a pipeline that turns 1,191 lecture videos into an aligned sequence of spoken words and pen movements (12.5M tokens), trained one small transformer on that sequence, and ran a controlled experiment that removes one kind of context at a time. The result: the board *does* help predict the next word (a robust +0.125 nats), but putting pen movements into the same sequence as the words costs more than that help is worth (−0.42 nats), and the speech does not help predict the pen at all. In plain terms: the two signals are related, but mixing them into one stream is the wrong way to use that relationship at this scale. That finding points to the next design, where a language agent plans what to say and what to draw, and a separate renderer puts it on the board.

1 The question

Watch The Organic Chemistry Tutor explain fractions. He says “two fifths plus one fourth” while writing $\frac{2}{5} + \frac{1}{4}$. The words and the pen are clearly related. My question was whether that relationship is something a model can *use*:

Does what is already on the board tell you the next word he will say? Does what he has said tell you the next pen movement?

If both are yes, one model can take the board and the speech as shared context and produce both. If not, the two should be handled separately. This report is the experiment that decides.

Nobody had tested this on real teaching video. The closest existing system plans whiteboard content with a vision-language model from 24 hand-made demonstrations and renders it in sync with narration [1]; it does not learn from a real teacher. So the first job was to build the data.

2 What others have done

Making a machine write by hand. The standard recipe is a sequence model that emits pen movements one point at a time, where each output is not a single point but a *mixture of Gaussians* over where the pen might go next [2]. Graves showed this produces convincing cursive when the model is also told which characters to write [3]; that work trained on IAM-OnDB, about 86k words from 200+ writers captured on a whiteboard [4]. Sketch-RNN applied the same output head to drawings, with the pen encoded as $(\Delta x, \Delta y, \text{pen state})$ [5] — the format this report uses. Later work separates

what is written from *how* it is written [6], or replaces the point-by-point model with diffusion over whole strokes [7, 8], or tokenizes ink so an ordinary language model can consume it [9], or skips the pen entirely and generates the act of sketching as video [10]. A dataset of 230k handwritten math expressions exists for recognition [11], but it has no speech and no timing.

Getting handwriting out of lecture video. Detecting and binarizing what a teacher wrote on a whiteboard in a fixed-camera lecture video is a studied problem [12, 13]. Those systems produce the final image on the board; this project additionally needs the *order and timing* of every stroke, so it can be lined up with the speech.

Putting two kinds of tokens in one transformer. Large multimodal models have gone both ways. Chameleon puts images and text into a single token sequence and trains on it from scratch at 7–34B parameters [14]. Flamingo keeps the language model’s sequence pure and lets it look at visual features through separate cross-attention layers [15]. The experiment in this report is, in miniature, a comparison of those two philosophies for speech and pen.

Language models that draw. A recent line of work has a language model output a *plan* for a picture — element types and coordinates — and renders it: diagrams [16], scene layouts [17], scientific figures as TikZ code [18], and sketches drawn stroke by stroke on a numbered grid with no training at all [19]. The next step proposed at the end of this report belongs to this family.

3 Data: turning videos into a training sequence

This was most of the work. No dataset of “what a teacher says and draws, aligned in time” exists, so I built one.

3.1 Pipeline

Each video goes through six stages (Figure 1):

1. **Download** the video and audio.
2. **Frames.** Extract frames at 30 fps.
3. **Stroke extraction.** Detect newly deposited ink between frames, thin it to a skeleton, and track the pen tip. Output: strokes as lists of (x, y, t) points, subsampled to 10 fps. Coordinates are normalized to $[0, 1]$ over the 1280×720 canvas. This is the same detection problem as in lecture-video summarization [12, 13], but keeping the temporal order of the ink rather than only the final image.
4. **Speech to text.** Whisper [20] on the audio track gives every word with a start and end time.
5. **Merge.** Words and strokes are interleaved into one time-ordered event stream.
6. **Align.** Each stroke is attached to the word being spoken while it was drawn (maximum time overlap). Strokes drawn while nobody is speaking are attached to a `<silent>` token.

```
video → frames (30 fps) → stroke extraction → strokes.jsonl
                        audio → Whisper → words.jsonl
strokes + words → merge → align → training.jsonl
```

Figure 1: The data pipeline. One command runs it end to end for a YouTube URL.

The pipeline ran across 8–12 lab machines in parallel. (Scaling to 30 machines triggered YouTube rate limiting, so 8–12 is the practical ceiling.) The manifest had 1,372 videos; 1,191 completed cleanly.

3.2 Why this teacher

The Organic Chemistry Tutor writes slowly and deliberately, one symbol at a time, with clear gaps between strokes. That makes extraction reliable. Cursive-style teachers (Khan Academy) were tried and deferred: the cursor and freshly deposited ink overlap at the pen tip, and the reconstructions were unreadable.

3.3 What the data looks like

Before training anything I measured the corpus, and several numbers changed the design (Table 1). For scale: the 9.7M pen points here are more than the whole of IAM-OnDB, the dataset the classic handwriting-synthesis results were trained on [3, 4], and they all come from one writer.

	value	why it matters
videos / pages	1,191 / 1,191	every video is one board; no board-clear events survived
total tokens	12.5M	
pen points	9.7M (78%)	the sequence is mostly pen
words	2.77M (22%)	
vocabulary	5,525	after lower-casing and stripping Whisper’s punctuation (w
median page	8,860 tokens	95% of pages are longer than a 1,024-token window
points per stroke	median 6	10 fps sampling; a letter is a handful of points
drawing that starts from <silent>	49%	half of all writing happens with nobody talking
within-stroke step	median 0.006	tiny
between-stroke jump	median 0.04, 90th pct 0.30	large; pen motion is two very different distributions

Table 1: The corpus after the pipeline. Distances are in normalized canvas units.

Two of these directly shaped the model. The bimodal pen motion (tiny steps within a stroke, big jumps between strokes) is why the output head is a mixture of Gaussians rather than a single predicted point. The 49% of drawing that starts from <silent> is why that token must be allowed at generation time — an earlier version banned it and could barely draw.

4 The model, in plain terms

One transformer [21] reads the lecture as a single sequence of tokens in the order they happened:

[today] [we] [have] [pen] [pen] [pen] [a] [squared] [pen] [<silent>] [pen] ...

A word token carries a word. A pen token carries where the pen moved (dx , dy) and whether it lifted, following the Sketch-RNN encoding [5]. At every position the model predicts what comes next. Since the next thing can be a word or a pen move, it has three small output heads: one that decides *which kind* comes next, one that picks the word, and one that says where the pen goes.

The pen head predicts a *distribution* of possible next moves (20 overlapping Gaussian blobs), not a single point — a mixture density network [2, 3]. If you only predict one point, you get the average of “curve left” and “curve right”, which is “stay still”, and the strokes collapse into dots. That is exactly what the earlier attempts on this project produced.

The model is small on purpose: 4.6M parameters, 20 epochs, about six minutes per training run on a free Colab GPU. The goal was a clean comparison between conditions, not a strong model.

Details for reproduction. 4 layers, model width 256, 8 attention heads, pre-norm layout [22], GELU activations [23], sinusoidal position encoding [21], context window 1,024 tokens. Trained with AdamW [24] at learning rate 3×10^{-4} with a 200-step warm-up and cosine decay, batch size 8, mixed precision [25], label smoothing 0.1 on the word head [26]. Pen coordinates are deltas scaled by 20 so their standard deviation (≈ 1.25) matches the mixture’s initial width.

How to read the numbers. Every result below is a *loss*: how surprised the model was by what actually came next, averaged over the held-out lectures. Lower is better. Word loss is in *nats*; e^{loss} is the *perplexity*, roughly “how many words the model was choosing between” — perplexity 47 means it had narrowed the next word down to about 47 candidates. Pen loss is also in nats but can be negative, because it is the log of a probability density rather than a probability. A difference of 0.1 nats between two conditions is large: it is the difference between perplexity 47 and 52. Every number is the mean over three separately trained models (three random seeds), and a result is only called *robust* when all three seeds agree.

5 How I measured it

The idea is simple: keep everything the same and take away one kind of context at a time, then see how much worse the prediction gets. To test “does the board help the next word”, the conditions are:

full	the model sees the words and the real pen movements
mask_strokes	the pen tokens are there, but their coordinates are zeroed out
nostroke_hidden	the pen tokens are there but hidden from attention entirely
drop_strokes	the pen tokens are removed; the model sees only words

Each step down removes one thing: first the pen *content*, then the fact that a pen event *happened*, then the empty *space* the pen tokens took up. Reading the word loss at each step tells you what each of those was worth. The same ladder in the other direction (**mask_words**, **noword_hidden**, **drop_words**) tells you what the speech is worth for predicting the pen.

All conditions are evaluated on exactly the same 363 held-out windows, so the comparison is fair.

5.1 Two things I had to fix along the way

The first version compared the wrong things. It only had **mask** and **drop**. But removing the pen tokens also shrinks the window to a quarter of its length and pushes the words right next to each other, so “drop minus mask” mixed two effects: losing the pen information, and changing the spacing. The **hidden** condition was added to separate them — same length, same spacing, pen slots simply invisible.

One training run is not enough. Training the same condition twice with the same seed gave losses of 4.22 and 4.36. GPU arithmetic in half precision is not deterministic, and a small model amplifies the difference. That the random seed alone can move results by more than the effect being studied is well documented [27, 28], and the 0.14-nat gap here is bigger than most of the effects I was measuring. So every condition was trained three times and I only trust a result all three agree on. Within each run, the uncertainty is a bootstrap [29] over held-out *windows* rather than tokens, because tokens from the same lecture are not independent.

6 Results

Figure 2 and Table 2 show the loss at every step of both ladders.

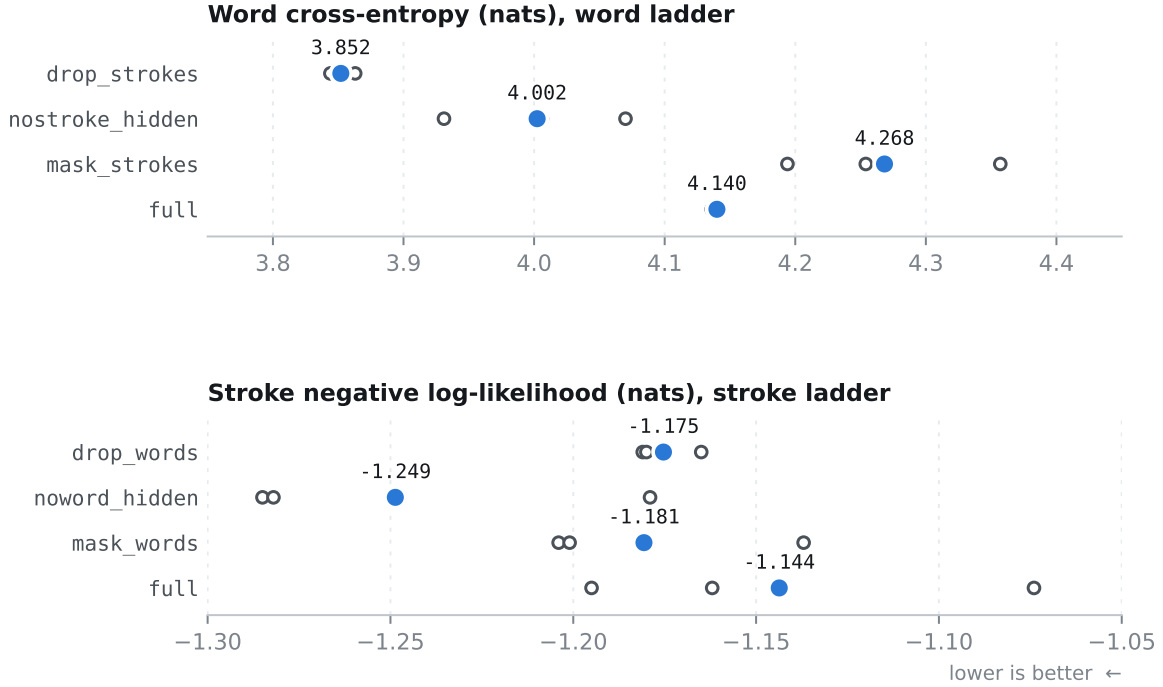


Figure 2: Loss at each condition. Filled marks are the mean of three seeds; hollow dots are the individual seeds. **Top:** the word ladder runs backwards — the less pen information the model sees, the better it predicts words. **Bottom:** the stroke ladder is nearly flat; the four conditions sit within 0.1 nats and the seeds overlap.

condition	word loss	perplexity	pen loss
full	4.140 ± 0.003	62.8	-1.144 ± 0.063
mask_strokes	4.268 ± 0.082	71.4	-0.741 ± 0.023
nostroke_hidden	4.002 ± 0.070	54.7	—
drop_strokes	3.852 ± 0.010	47.1	—
mask_words	4.384 ± 0.002	80.2	-1.181 ± 0.038
noword_hidden	—	—	-1.249 ± 0.061
drop_words	—	—	-1.175 ± 0.009

Table 2: Held-out loss, mean $\pm$ std over three seeds. The word-only model (**drop_strokes**) predicts words best.

The numbers that answer the question are the *differences between neighbouring steps*, because each one isolates a single effect. Figure 3 and Table 3 show them as “how much the loss went down when this information was added”, so a positive number means it helped.

7 What it means

1. Yes — the board tells you the next word. Giving the model the real pen coordinates instead of zeros makes its word predictions better by 0.125 nats, in every seed. This is the cleanest comparison in the whole design: the two sequences are identical token for token, only the pen numbers differ.

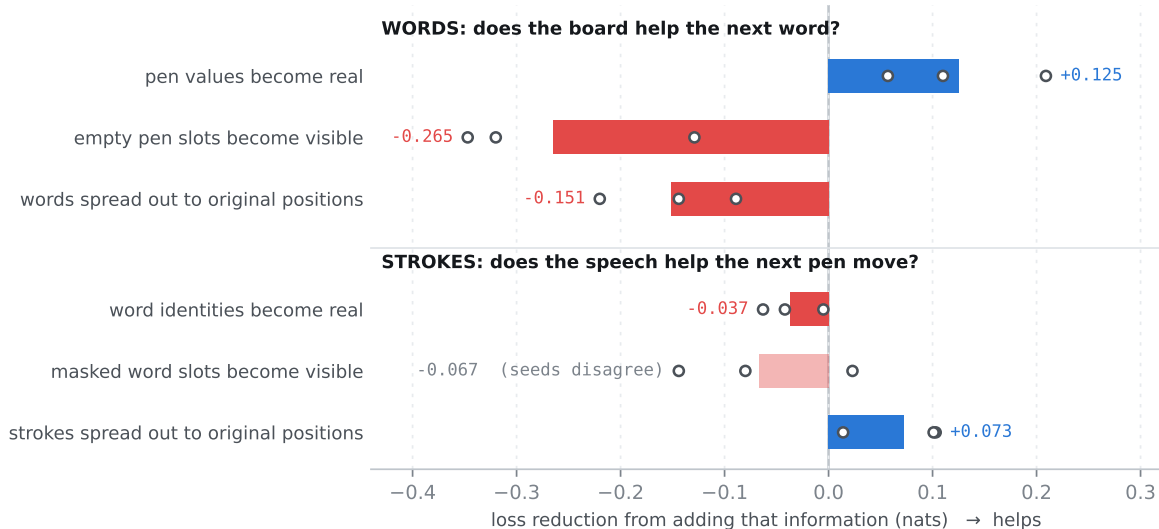


Figure 3: What each kind of context is worth. Blue bars helped, red bars hurt. Hollow dots are the three seeds. Every bar except one has all three seeds on the same side of zero.

what was added	Δ (nats)	per seed	verdict
<i>Words: does the board help the next word?</i>			
real pen coordinates	$+0.125 \pm 0.077$	+0.21/ +0.06/ +0.11	robust, helps
empty pen slots become visible	-0.266 ± 0.119	-0.35/ -0.13/ -0.32	robust, hurts
words spread to original spacing	-0.151 ± 0.066	-0.14/ -0.22/ -0.09	robust, hurts
<i>Strokes: does the speech help the next pen move?</i>			
real word identities	-0.037 ± 0.029	-0.04/ -0.01/ -0.06	robust, hurts a little
masked word slots become visible	-0.067 ± 0.084	-0.08/ +0.02/ -0.14	seeds disagree
strokes spread to original spacing	$+0.073 \pm 0.051$	+0.10/ +0.01/ +0.10	robust, but see §9

Table 3: Each step of the ladder as a loss reduction. *Robust* means all three seeds agree in sign and each seed’s confidence interval (bootstrapped over held-out windows) excludes zero.

When the pen is drawing a fraction bar, “over” becomes more likely, and the model learned that.

2. But putting the pen in the same sequence costs more than that. Two separate costs showed up. Simply having the pen slots *visible* — even with no information in them — costs 0.266 nats. Attention spreads across everything it can see, and when 78% of the sequence is pen tokens, most of it is spent on tokens that say nothing about the next word. A small model does not fully learn to ignore them. This is the same phenomenon seen in large language models, which get measurably worse at a task when irrelevant material is added to the prompt [30] or when the useful part of a long context is buried among the rest [31]. On top of that, the pen tokens push the words apart, and that spacing alone costs another 0.151 nats: with sinusoidal absolute positions [21], “the previous word” is a relation the model has to re-learn at every distance, which is one reason later transformers moved to relative or rotary position encodings [32, 33]. Add it up: $+0.125 - 0.266 - 0.151 = -0.29$. A model that never sees the pen predicts words *better* than the full model (perplexity 47 vs. 63).

3. No — the speech does not tell you the next pen move. Giving the model the real words instead of a placeholder makes pen prediction slightly *worse*, in all three seeds. Where the pen goes next is decided by where it just was; the speech does not get in.

Put together: the board and the voice are genuinely related, in one direction. But the way this model used the relationship — one flat sequence with both kinds of tokens, the Chameleon-style choice [14] — loses more than it gains at this size. “More context is better” stops being true when the extra context takes up 78% of what the model is looking at.

8 What the model could not do

It could not draw. Given 400 tokens of a real lecture and asked to continue, it produces word salad and pen scatter (Figures 4 and 5). The scatter lands in roughly the right part of the board — it learned *where* writing goes — but never forms a digit or a letter.

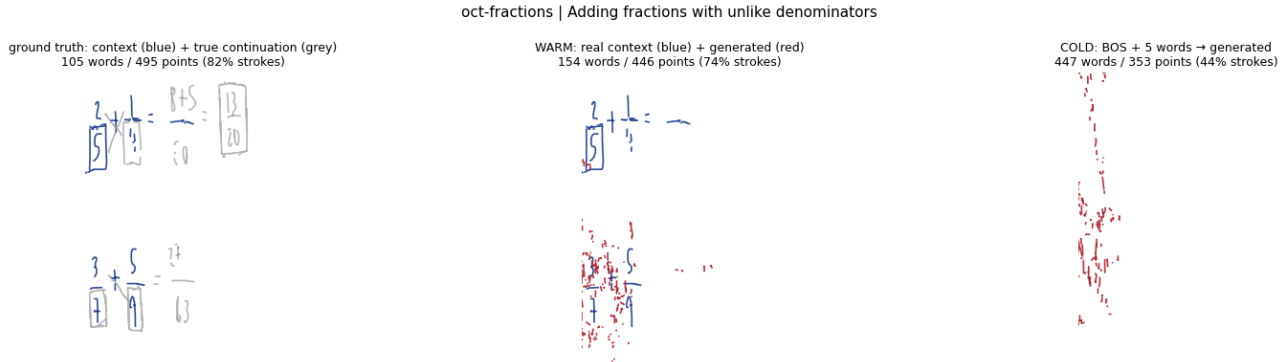


Figure 4: *Adding fractions with unlike denominators*. Left: the real lecture — context in blue, what actually came next in grey (two fraction problems with boxed answers). Middle: the same real context, then 600 tokens generated by the model in red. The scatter sits where the second problem really goes, but nothing in it is a digit. Right: generating from just the first five words.

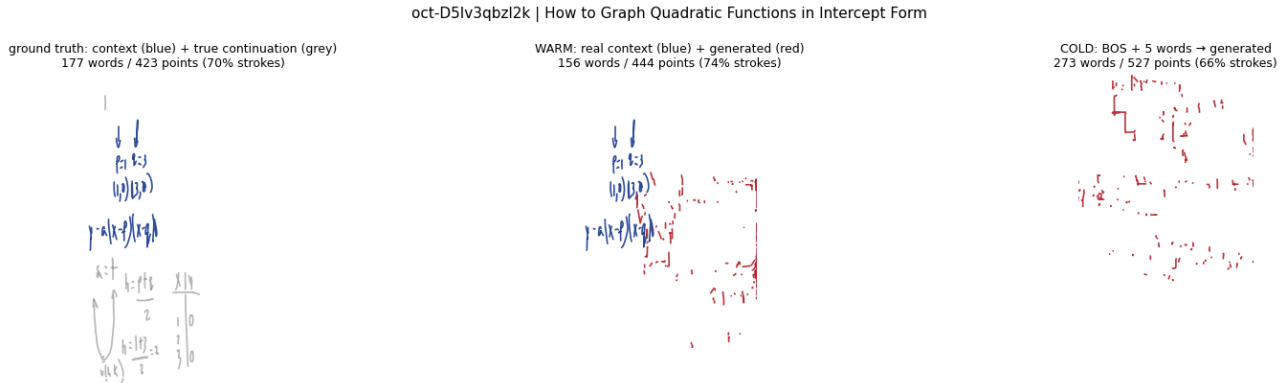


Figure 5: *Graphing quadratic functions in intercept form*. Same layout. The real page has an equation, a vertex formula and a table of x/y values; the generated page has neither a curve nor a character.

Part of this is the usual gap between predicting one step with the true history and generating hundreds of steps from your own output: small errors feed back and compound [34, 35]. But the bigger reason is what the data does not contain. It is not that the data is too small: pen loss was still falling at the last epoch in all 21 runs, and the corpus is larger than what the classic handwriting models were trained on. What the data lacks is a label saying *which symbol is being drawn*. Graves’ model produced readable cursive because it was told the character string and attended to it while writing [3]; the systems that separate content from style make the same point from the other side [6]. Here the only clue to the

content is the speech, and finding 3 says the speech is not a usable clue at this size. The model learned how a pen moves, not what it is writing.

9 Limitations

1. **Small model.** 4.6M parameters. A larger model could learn to ignore the pen tokens, shrinking cost 2 until benefit 1 wins — Chameleon reports that early fusion works at 7B and above [14]. All results are “at this scale”.
2. **One rung of the stroke ladder is not perfectly clean.** The `<silent>` token stays visible in `mask_words` and `noword_hidden` but is removed in `drop_words`, so the last row of Table 3 includes the value of that marker, not just spacing.
3. **Seed spread.** The three seeds agree on sign and their intervals do not overlap zero, but the magnitudes vary 2–3 $\times$. The ranges are more honest than the means.
4. **Alignment noise.** Attaching strokes to words by time overlap is approximate. Some pairs are wrong, which would make finding 1 look smaller and finding 3 look closer to zero than they really are.
5. **No board clears.** Every video is treated as one page, so “where things are on the board” is only meaningful locally.

10 Next: an agent that plans what to say and what to draw

The experiment says two things about how to build the real system. The information linking speech and drawing is real (finding 1), and the data is missing the one thing a drawing model needs most — an explicit statement of *what* to draw (finding 3 and the generation failure). A flat stroke model tries to discover that from correlation and cannot. So the next design supplies it directly.

The plan is a **language-model agent that writes the lecture as a script**: for each step, what to say, and what to put on the board — a shape, an equation, a diagram, a label — with where to put it. Something like:

say	draw
“Let’s add two fifths and one fourth.”	<code>fraction 2/5 at (0.15, 0.20); symbol + at (0.27, 0.22)</code>
“The denominators are different, so we need a common one.”	<code>underline denominators; text “LCD = 20” at (0.15, 0.4)</code>
“Five times four is twenty.”	<code>arrow from (0.17, 0.26) to (0.20, 0.38)</code>

The agent decides the content and the layout; a renderer turns the drawing commands into strokes on a canvas. This is the pattern that has worked for diagrams [16], page layouts [17] and scientific figures [18], and SketchAgent showed that a frontier model can place strokes on a coordinate grid competently when the coordinates are given as a small vocabulary rather than raw numbers [19]. Running it as a loop — plan a step, render it, look at the board, plan the next [36] — gives the agent the visual memory of the board that the flat model never had. It also separates the two jobs the flat model was forced to do at once, which is exactly what finding 2 says to do: keep the language reasoning in a stream that is all language, and give the drawing side an explicit instruction instead of asking it to guess from the words — the Flamingo-style division of labour [15] rather than the Chameleon-style one.

What the dataset from Section 3 contributes to this: the aligned transcripts are the examples of *how* this teacher structures an explanation — what he says first, when he pauses to write, how much he draws per sentence — and the extracted strokes are the reference for the layout conventions (where problems start, how far apart lines go) and, later, for rendering the primitives in his handwriting rather than a font, where the current text-to-ink models [7] are the natural tool. The evaluation would ask two things the current model could not be asked: does the drawing match what is being said, and does the layout stay readable as the lecture goes on.

Experiment numbers from `ctx_ablation_v2/` (7 conditions $\times$ 3 seeds, 363 held-out windows, 3,000 bootstrap resamples). The full design, the first run, and every bug found along the way are documented in `METHOD1_CONTEXT_EXPERIMENT.md`.

References

- [1] Suraj Prasad and Pinak Mahapatra. Speech-synchronized whiteboard generation via VLM-driven structured drawing representations. *arXiv preprint arXiv:2603.25870*, 2026.
- [2] Christopher M. Bishop. Mixture density networks. Technical Report NCRG/94/004, Aston University, Neural Computing Research Group, 1994.
- [3] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [4] Marcus Liwicki and Horst Bunke. IAM-OnDB – an on-line English sentence database acquired from handwritten text on a whiteboard. In *Proceedings of the Eighth International Conference on Document Analysis and Recognition (ICDAR)*, pages 956–961, 2005. doi: 10.1109/ICDAR.2005.132.
- [5] David Ha and Douglas Eck. A neural representation of sketch drawings. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1704.03477.
- [6] Emre Aksan, Fabrizio Pece, and Otmar Hilliges. DeepWriting: Making digital ink editable via deep generative modeling. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018. doi: 10.1145/3173574.3173779.
- [7] Wei Pan, Huiguo He, Hiuyi Cheng, Yilin Shi, and Lianwen Jin. DiffInk: Glyph- and style-aware latent diffusion transformer for text to online handwriting generation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.23624.
- [8] Jin Zhou, Yi Zhou, Hongliang Yang, Pengfei Xu, and Hui Huang. StrokeFusion: Vector sketch generation via joint stroke-UDF encoding and latent sequence diffusion. *arXiv preprint arXiv:2503.23752*, 2025.
- [9] Douglass Wang. ScribeTokens: Fixed-vocabulary tokenization of digital ink. *arXiv preprint arXiv:2603.02805*, 2026.
- [10] Hui Ren, Yuval Alaluf, Omer Bar Tal, Alexander Schwing, Antonio Torralba, and Yael Vinker. VideoSketcher: Sequential sketch generation using video model priors. *arXiv preprint arXiv:2602.15819*, 2026.
- [11] Philippe Gervais, Anastasiia Fadeeva, and Andrii Maksai. MathWriting: A dataset for handwritten mathematical expression recognition. *arXiv preprint arXiv:2404.10690*, 2024.

- [12] Kenny Davila and Richard Zanibbi. Whiteboard video summarization via spatio-temporal conflict minimization. In *14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, pages 355–362, 2017.
- [13] Bhargava Urala Kota, Kenny Davila, Alexander Stone, Srirangaraj Setlur, and Venu Govindaraju. Generalized framework for summarization of fixed-camera lecture videos by detecting and binarizing handwritten content. *International Journal on Document Analysis and Recognition (IJDAR)*, 22(3), 2019. doi: 10.1007/s10032-019-00327-y.
- [14] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.
- [15] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] Abhay Zala, Han Lin, Jaemin Cho, and Mohit Bansal. DiagrammerGPT: Generating open-domain, open-platform diagrams via LLM planning. In *Conference on Language Modeling (COLM)*, 2024.
- [17] Weixi Feng, Wanrong Zhu, Tsu-jui Fu, Varun Jampani, Arjun Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and William Yang Wang. LayoutGPT: Compositional visual planning and generation with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [18] Jonas Belouadi, Anne Lauscher, and Steffen Eger. AutomaTikZ: Text-guided synthesis of scientific vector graphics with TikZ. In *International Conference on Learning Representations (ICLR)*, 2024.
- [19] Yael Vinker, Tamar Rott Shaham, Kristine Zheng, Alex Zhao, Judith E. Fan, and Antonio Torralba. SketchAgent: Language-driven sequential sketch generation. *arXiv preprint arXiv:2411.17673*, 2024.
- [20] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2212.04356.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [22] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.
- [23] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [24] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [25] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.

- [26] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [27] David Picard. Torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision. *arXiv preprint arXiv:2109.08203*, 2021.
- [28] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [29] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979.
- [30] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning (ICML)*, 2023.
- [31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.
- [32] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of NAACL-HLT*, 2018.
- [33] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [34] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [35] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.